

Table des matières

1	Présentation et fonctionnement de l'agent IA	6
1.1	Introduction	6
1.2	Analyse de la Classe CodeAnalysisAgent	6
1.2.1	Importation des bibliothèques	6
1.2.2	Initialisation de l'Agent (__init__)	7
1.2.3	Configuration du Répertoire (_setup_report_dir)	8
1.2.4	Initialisation du LLM (_init_llm)	9
1.2.5	Nettoyage du Texte (_clean_text)	10
1.2.6	Détection de Langue (_detect_language)	10
1.2.7	Création Prompt Docstrings (_create_docstring_prompt)	10
1.2.8	Création Prompt Commentaires (_create_comment_prompt)	12
1.2.9	Analyse du Projet (analyze_project)	14
1.2.10	Scan des Fichiers (_scan_files)	16
1.2.11	Analyse de l'Implémentation (_analyze_implementation)	17
1.2.12	Analyse des Tests (_analyze_tests)	20
1.2.13	Vérification des Documentations (_verify_docstring)	22
1.2.14	Vérification des Commentaires (_verify_comment)	23
1.2.15	Vérification des Tests (_verify_test_file)	24
1.2.16	Appel au LLM (_call_llm)	25
1.2.17	Sauvegarde Réponses Échouées (_save_failed_response)	28

1.2.18	Calcul des Métriques (<code>_compute_metrics</code>)	28
1.2.19	Rendu Rapport Terminal (<code>_render_terminal_report</code>)	29
1.2.20	Sauvegarde du Rapport (<code>_save_report</code>)	32
1.2.21	Vérification Fichier Test (<code>_is_test_file</code>)	33
1.2.22	Bloc Principal (<code>__main__</code>)	34
1.3	Analyse de la Classe <code>TestAnalyzer</code>	35
1.3.1	Importation des bibliothèques	35
1.3.2	Initialisation de l'Analyseur (<code>__init__</code>)	35
1.3.3	Initialisation du LLM (<code>_init_llm</code>)	36
1.3.4	Analyse Statique Tests (<code>parse_test_file</code>)	37
1.3.5	Vérification des Assertions (<code>check_assertions</code>)	39
1.3.6	Analyse Sémantique Tests (<code>analyze_test_semantics</code>)	41
1.3.7	Analyse Complète Tests (<code>analyze</code>)	44
1.3.8	Sélection Fichier Analyse (<code>select_and_analyze</code>)	46
1.4	Conclusion	47
2	Évaluation de l'agent	48
2.1	Introduction	48
2.2	preparation des testes	48
2.3	Calcule de score	51
2.4	Conclusion	53

Table des figures

2.1	Le dossier Testes	48
2.2	Le dossier T1	49
2.3	Le dossier json	49
2.4	Le document doc1.json	50
2.5	Le dossier reports	51
2.6	Le score	53

Listings

1.1	Importation des bibliothèques	7
1.2	Initialisation de l'Agent	8
1.3	Configuration du répertoire	8
1.4	Initialisation du LLM	9
1.5	Nettoyage du texte	10
1.6	Détection de Langue	10
1.7	Création du prompt afin d'analyser les documentations fournis	10
1.8	Création prompt commentaires	13
1.9	Analyse du projet	15
1.10	Scan des fichiers	16
1.11	Analyse de l'implémentation	18
1.12	Analyse des tests	21
1.13	Vérification des documentations	23
1.14	Vérification des commentaires	23
1.15	Vérification des tests unitaires	24
1.16	Appeler le LLM	25
1.17	Sauvgardage des réponses échoués	28
1.18	Calcul des métriques	29
1.19	Rapport Terminal	30
1.20	Sauvegarde des rapports sous forme .JSON	33

1.21	Vérification du fichier test	34
1.22	Programme Principal	34
1.23	Importation des bibliothèques	35
1.24	Initialisation de l'Analyseur	36
1.25	Initialisation du LLM	36
1.26	Analyse des tests	37
1.27	Vérification des assertions	40
1.28	L'Analyse complète des tests	44
1.29	Méthode de selection des fichiers de tests avec Python	46
2.1	Calcule de score	52

Chapitre 1

Présentation et fonctionnement de l'agent IA

1.1 Introduction

Ce chapitre expose le fonctionnement détaillé d'un agent IA dédié à l'analyse de projets Python, nommé `CodeAnalysisAgent`. Conçu pour détecter les lacunes documentaires et structurelles dans le code source. Cet agent combine deux approches complémentaires : l'analyse statique à l'aide de l'arbre syntaxique abstrait (AST) et l'évaluation sémantique basée sur un modèle de langage avancé (LLM). Grâce à cette double stratégie, l'outil peut identifier des fonctions non documentées, des commentaires imprécis ou encore des tests unitaires incomplets ou absents. L'ensemble du processus est entièrement automatisé, de la détection des fichiers jusqu'à la génération d'un rapport structuré. En complément, la classe `TestAnalyzer` permet une analyse approfondie des tests unitaires

1.2 Analyse de la Classe `CodeAnalysisAgent`

1.2.1 Importation des bibliothèques

— **Modules standards Python :**

- `os`, `time`, `json`, `re`, `ast`, `tokenize`, `unicodedata`, `BytesIO` : utilisés pour la manipulation des chemins, du texte, des expressions régulières, des flux binaires et de l'AST Python. (AST (Abstract Syntax Tree) en Python est une représentation en arbre du code source permettant son analyse ou sa transformation avant exécution.)
- `typing` : fournit des annotations de type pour améliorer la lisibilité et la sécurité du code (`Dict`, `List`, `Tuple`, `Optional`).
- `pathlib.Path` : simplifie la gestion des chemins de fichiers de manière orientée objet.

— **Bibliothèques tierces :**

- `dotenv.load_dotenv` : permet de charger des variables d'environnement à partir du fichier `.env`, utile pour sécuriser les clés d'API.
- `google.api_core.exceptions` : Utilisé afin de gérer les erreurs liées aux appels aux API Google.
- `langchain_google_genai.ChatGoogleGenerativeAI` : permet d'interagir avec le modèle de langage génératif Gemini via l'API LangChain. (LangChain : Framework qui facilite l'intégration des modèles de langage volumineux dans les applications)
- `langchain_core.prompts.PromptTemplate` : sert à construire dynamiquement des prompts structurés pour les appels au LLM.
- `langdetect.detect` : détection automatique de la langue d'un texte pour adapter l'évaluation des commentaires ou docstrings.

Cette section prépare tous les outils nécessaires à l'analyse statique du code, à l'évaluation linguistique et à l'interaction avec le LLM externe utilisé.

```
import os
import ast
import json
import re
import time
from pathlib import Path
from typing import Dict, List, Tuple, Optional
from dotenv import load_dotenv
import tokenize
from io import BytesIO
import google.api_core.exceptions
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import PromptTemplate
from langdetect import detect
import unicodedata
```

Listing 1.1 – Importation des bibliothèques

1.2.2 Initialisation de l'Agent (`__init__`)

Cette méthode a pour but de préparer tous les éléments nécessaires pour que l'agent fonctionne correctement,

- `self.metrics_template` : initialise les métriques qu'on utilisera lors de l'analyse.
- `self.report_dir` : convertit le chemin de dossier fourni en un objet Path de la bibliothèque Pathlib afin de simplifier sa manipulation

- `self._setup_report_dir` : Consiste à créer le dossier de rapports s'il n'existe pas afin de prévenir toute sorte de problèmes qu'on peut rencontrer
- `self.llm` initialise le modèle de langage utilisé pour l'analyse.
- `self.docstring_prompt`, `self.comment_prompt` et `self.test_prompt` préparent les prompts pour l'évaluation des docstrings, des commentaires et des tests.

```
def __init__(self, report_dir: str =
    r"C:\Users\kordoghli\Desktop\AI\Reports"):
    self.metrics_template = {
        "files_exist": 0,
        "files_valid": 0,
        "has_tests": 0,
        "tests_valid": 0,
        "fully_documented": 0,
        "all_functions_commented": 0,
        "docstrings_accurate": 0.0,
        "comments_accurate": 0.0
    }
    self.report_dir = Path(report_dir)
    self._setup_report_dir()
    self.llm = self._init_llm()
    self.docstring_prompt = self._create_docstring_prompt()
    self.comment_prompt = self._create_comment_prompt()
    self.test_prompt = self._create_test_prompt()
```

Listing 1.2 – Initialisation de l'Agent

1.2.3 Configuration du Répertoire (`_setup_report_dir`)

Cette méthode est nécessaire pour créer une répertoire pour les rapports

- `self.report_dir.mkdir(parents=True, exist_ok=True)` tente de créer le dossier de rapports, y compris les dossiers parents si nécessaire.
- `except Exception as e` capture toute erreur pouvant survenir lors de la création du dossier.
- `self.report_dir = Path.cwd()`, cette fonction réinitialise le répertoire de sortie vers le répertoire courant.

```
def _setup_report_dir(self) -> None:
    try:
        self.report_dir.mkdir(parents=True, exist_ok=True)
        print(f"R pertoire rapports : {self.report_dir}")
    except Exception as e:
```



```

print(f"  chec  cr ation r pertoire {self.report_dir}: {e}")
self.report_dir = Path.cwd()
print(f"Utilisation r pertoire courant : {self.report_dir}")

```

Listing 1.3 – Configuration du répertoire

1.2.4 Initialisation du LLM (_init_llm)

La fonction `_init_llm` tente d'initialiser le modèle de langage Gemini de Google, en procédant comme suit :

- `api_key = os.getenv("GOOGLE_API_KEY")` récupère la clé API situé dans le fichier `.env`
- `if not api_key: raise ValueError(...)` vérifie que la clé existe, sinon elle provoque une exception qui va être affichée comme un message d'erreur.
- `llm = ChatGoogleGenerativeAI(...)` initialise l'agent LLM avec le modèle Gemini 2.0 Flash Lite en utilisant le "API Key" récupérée dans le fichier `.env`.
- `print(...)` affiche un message confirmant la réussite de l'initialisation.
- `return llm` retourne l'objet LLM initialisé pour la prochaine utilisation.
- `except Exception as e` intercepte toute erreur survenue pendant l'initialisation.
- `print(...)` affiche un message d'erreur si l'initialisation échoue.
- `return "None"` retourne `None` afin d'indiquer que le LLM n'a pas pu être initialisé avec succès.

```

def _init_llm(self) -> Optional[ChatGoogleGenerativeAI]:
    try:
        api_key = os.getenv("GOOGLE_API_KEY")
        if not api_key:
            raise ValueError("GOOGLE_API_KEY manquant")
        llm = ChatGoogleGenerativeAI(
            model="gemini-2.0-flash-lite",
            google_api_key=api_key,
            temperature=0.3
        )
        print("LLM initialisé (Gemini 2.0 Flash Lite)")
        return llm
    except Exception as e:
        print(f"  chec  initialisation LLM: {e}")
        return None

```

Listing 1.4 – Initialisation du LLM

1.2.5 Nettoyage du Texte (_clean_text)

La fonction `_clean_text` sert à nettoyer le code à analyser afin de le rendre plus exploitable et systématique pour l'analyse en utilisant `unicodedata.normalize().encode().decode` pour transformer les caractères accentués (exemple : é, â, û) ou spéciaux (exemple : £, ¸, ×) en leur version ASCII et supprimer les espaces et les caractères non imprimables. Cette procédure est nécessaire pour assurer une analyse sans erreurs

```
def _clean_text(self, text: str) -> str:
    text = unicodedata.normalize('NFKD', text).encode('ascii',
        'ignore').decode('ascii')
    text = ''.join(c for c in text if c.isprintable())
    return text.strip()
```

Listing 1.5 – Nettoyage du texte

1.2.6 Détection de Langue (_detect_language)

Cette méthode consiste à détecter la langue utilisée lors de l'écriture des commentaires et documentations.

```
def _detect_language(self, text: str) -> str:
    try:
        return detect(text)
    except:
        return "en" # Par d faut : anglais
```

Listing 1.6 – Détection de Langue

1.2.7 Création Prompt Docstrings (_create_docstring_prompt)

La fonction `_create_docstring_prompt` construit un prompt destiné au modèle LLM déjà initialisé afin de vérifier la pertinence de la documentation élaboré et si elle respecte les politiques déjà soulignés

```
def _create_docstring_prompt(self) -> PromptTemplate:
    return PromptTemplate(
        input_variables=["func_name", "docstring", "code", "lang"],
        template="""V rifie si la docstring, crite en langue
            '{lang}', d crit pr cis ment le comportement de la
            fonction Python suivante au moins 65% de pr cision. La
            description doit correspondre exactement au comportement du
            code, sans incoh rences majeures (par exemple, d crire une

```

```

        addition pour une soustraction est inacceptable).  value
        UNIQUEMENT la docstring fournie, sans tenir compte des
        commentaires ou d'autres lments du code. N'invente pas
        d'erreurs absentes du texte de la docstring.

Retourne UNIQUEMENT un objet JSON avec :
- "is_accurate": true si la docstring est pr cise 65% ou plus, false
  sinon
- "issues": liste des probl mes sp cifiques dans la docstring (vide si
  pr cise)
- "reason": explication d taill e expliquant pourquoi la docstring est
  jug e pr cise ou non, dans la langue de la docstring ou en anglais
  si ind termin e

Nom : {func_name}
Docstring : {docstring}
Code : {code}

**Retourne UNIQUEMENT l'objet JSON.**"""
    )

```

Listing 1.7 – Création du prompt afin d'analyser les documentations fournis

Explication du Prompt

Cette fonction construit un prompt destiné à un modèle de langage pour qu'il puisse évaluer la pertinence d'une documentation en Python. Le prompt suit des instructions strictes et utilise un format JSON comme unique réponse attendue. Les éléments suivants composent ce processus :

- **But principal** : Vérifier que la documentation d'une fonction décrit son comportement réel avec une précision d'au moins 65% (Le seuil d'acceptabilité d'une documentation) , sans contradiction avec le code source.
- **Variables d'entrée** : Le prompt est paramétré avec les éléments suivants :
 - `func_name` : nom de la fonction à évaluer.
 - `docstring` : texte de documentation de la fonction.
 - `code` : code source de la fonction concernée.
 - `lang` : langue dans laquelle la documentation est rédigée.
- **Instructions d'évaluation** :
 - La documentation doit correspondre exactement au comportement du code.

- Toute incohérence majeure (comme une mauvaise description de l'opération effectuée) entraîne une évaluation négative : sa précision sera attribué 0.
- L'analyse se fait uniquement sur la documentation, sans tenir compte des commentaires ou d'autres parties du code.
- Le modèle ne doit pas supposer des erreurs qui n'apparaissent pas dans la documentation.
- **Structure de la réponse** : Le modèle doit répondre uniquement avec un objet JSON contenant :
 - "is_accurate" : booléen indiquant si la documentation est jugée suffisamment précise.
 - "issues" : liste des erreurs ou incohérences détectées.
 - "reason" : justification détaillée de "issues", rédigée dans la langue détectée de la documentation si possible.
- **Texte final du prompt** : Comprend les champs (nom, docstring et code) suivis d'une consigne finale exigeant uniquement un retour JSON sans aucun ajout.
- **Utilité de fichier JSON** : Le choix d'un retour strictement structuré au format JSON peut être expliqué dans quelques points :
 1. **Structuration unique des réponses** : En obligeant un format JSON unique, on garantit que chaque évaluation suit une structure prévisible
 2. **Facilité d'exploitation automatique** : Les résultats peuvent être intégrés dans n'importe quel outil sans intervention humaine
 3. **Détection et gestion des erreurs** : Les erreurs deviennent simples à récupérer et filtrer
 4. **Clarté et traçabilité des évaluations** : Ce process dissocie clairement les résultats (is_accurate, issues, reason), ce qui améliore la transparence de l'analyse et la traçabilité des décisions prises par le LLM.

1.2.8 Création Prompt Commentaires (_create_comment_prompt)

La fonction `_create_comment_prompt` a pour but de générer un prompt destiné au LLM pour l'évaluation de commentaires Python associés à des fonctions. Elle construit un message structuré contenant :

- Le nom de la fonction analysée ;
- Le commentaire à évaluer ;
- Le code source de la fonction ;
- La langue dans laquelle le commentaire est rédigé.

En sortie, le modèle doit retourner un objet JSON contenant :

- Un booléen `is_accurate` indiquant si le commentaire est suffisamment précis ;
- Une liste `issues` des erreurs ou imprécisions détectés ;
- Une justification textuelle des `issues` dans `reason` expliquant la décision.

```

def _create_comment_prompt(self) -> PromptTemplate:
    return PromptTemplate(
        input_variables=["func_name", "comment", "code", "lang"],
        template="""V rifie si le commentaire, crit en langue
            '{lang}', d crit pr cis ment le comportement ou l'objectif
            de la fonction Python suivante au moins 65% de pr cision.
            La description doit correspondre exactement au comportement
            du code, sans incoh rences majeures (par exemple, d crire
            une addition pour une soustraction est inacceptable). value
            UNIQUEMENT le commentaire fourni, sans tenir compte de la
            docstring ou d'autres lments du code. N'invente pas
            d'erreurs absentes du texte du commentaire.

Retourne UNIQUEMENT un objet JSON avec :
- "is_accurate": true si le commentaire est pr cis 65% ou plus,
  false sinon
- "issues": liste des aspects manquants ou incorrects dans le
  commentaire (vide si pr cis)
- "reason": explication d taill e expliquant pourquoi le commentaire
  est jug pr cis ou non, dans la langue du commentaire ou en anglais
  si ind termin e

Nom : {func_name}
Commentaire : {comment}
Code : {code}

**Retourne UNIQUEMENT l'objet JSON.**""")

```

Listing 1.8 – Création prompt commentaires

Explication du Prompt Cette fonction permet de générer un prompt destiné au LLM , dont l'objectif est de vérifier la pertinence d'un commentaire associé à une fonction Python. Le prompt impose une structure précise de réponse sous forme d'objet JSON. Les détails de cette fonction sont les suivants :

- **Objectif principal** : Déterminer si le commentaire fourni décrit le rôle ou le comportement de la fonction, avec un taux de précision d'au moins 65%.
- **Variables utilisées dans le prompt** :
 - `func_name` : nom de la fonction concernée.
 - `comment` : commentaire en question à analyser.
 - `code` : code source complet de la fonction.

- `lang` : langue dans laquelle le commentaire est rédigé.
- **Instructions imposées au modèle :**
 - Le commentaire doit correspondre exactement au contenu du code.
 - Toute description incohérente (ex. confusion entre soustraction et addition) est considérée comme une erreur : Précision de ce commentaire sera attribué 0.
 - L'évaluation porte uniquement sur le commentaire, sans considérer la documentation ou d'autres parties du code.
 - Le modèle ne doit pas supposer des erreurs qui n'apparaissent pas dans le commentaire en question d'analyse.
- **Format attendu de la réponse :** Le modèle doit retourner uniquement un objet JSON contenant :
 - `"is_accurate"` : Booléen indiquant la précision du commentaire.
 - `"issues"` : liste des points imprécis ou incorrects (liste vide si le commentaire est correct).
 - `"reason"` : justification détaillée de `issues`, en langue d'origine du commentaire si possible.
 - **Contenu final du prompt :** Il inclut les informations (nom de la fonction, commentaire, code source), suivis d'une consigne finale exigeant uniquement un retour JSON sans aucun ajout.

1.2.9 Analyse du Projet (`analyze_project`)

La fonction `analyze_project` est responsable de l'analyse complète de code source situé dans le dossier donné. Elle s'exécute en deux phases principales et génère un rapport structuré contenant des métriques globales.

- **Initialisation du rapport :** Création d'une structure contenant les résultats de la phase 1 (scan de fichiers), de la phase 2 (analyse), ainsi qu'une copie vierge des métriques globales.
- **Phase 1 – Détection des fichiers :** La méthode `_scan_files` identifie les fichiers d'implémentation et de test présents dans le dossier. Les erreurs de cette phase sont stockées dans la clé `errors`.
- **Phase 2 – Analyse détaillée :**
 - Si un fichier d'implémentation est détecté, il est transmis à `_analyze_implementation`, qui extrait les fonctions, documentations, commentaires, et évalue leur précision.
 - Si un fichier de test est trouvé, il est analysé via `_analyze_tests` pour vérifier la validité des cas de test.
 - Les résultats sont enregistrés dans `phase2`, incluant les fonctions non commentées, non documentées, et les problèmes relevés dans les docstrings et commentaires.

- **Calcul des métriques** : L'ensemble des résultats est résumé dans une structure `combined_metrics`, produite par la méthode `_compute_metrics`.
- **Affichage et sauvegarde** : Un rapport est affiché dans le terminal à l'aide de `_render_terminal_report`, puis sauvegardé sur disque via `_save_report`. En cas d'exception, une erreur est capturée et ajoutée au rapport.

Cette fonction agit comme un contrôleur principal, orchestrant l'analyse complète du dossier de projet, et structurant les résultats pour une interprétation humaine ou automatique.

```
def analyze_project(self, folder_path: str) -> Dict:
    path = Path(folder_path)
    report = {
        "phase1": {"files_found": {}, "errors": []},
        "phase2": {
            "implementation_analysis": {},
            "test_analysis": {},
            "uncommented_functions": [],
            "undocumented_functions": [],
            "docstring_issues": {},
            "comment_issues": {}
        },
        "combined_metrics": self.metrics_template.copy()
    }

    print(f"Analyse dossier : {folder_path}")
    try:
        report["phase1"] = self._scan_files(path)
        if impl_file :=
            report["phase1"]["files_found"].get("implementation"):
            impl_report = self._analyze_implementation(Path(impl_file))
            report["phase2"]["implementation_analysis"] = impl_report
            report["phase2"]["uncommented_functions"] =
                impl_report.get("uncommented", [])
            report["phase2"]["undocumented_functions"] =
                impl_report.get("missing_docs", [])
            report["phase2"]["docstring_issues"] =
                impl_report.get("docstring_issues", {})
            report["phase2"]["comment_issues"] =
                impl_report.get("comment_issues", {})

        if test_file := report["phase1"]["files_found"].get("test"):
            report["phase2"]["test_analysis"] =
                self._analyze_tests(Path(test_file))
```

```

        report["combined_metrics"] = self._compute_metrics(report)
    print("\n" + self._render_terminal_report(report))
    self._save_report(folder_path, report)
except Exception as e:
    report["phase1"]["errors"].append(f"Erreur analyse : {e}")
    print("\n" + self._render_terminal_report(report))
    self._save_report(folder_path, report)
    print(f"Exception analyse : {e}")

return report["combined_metrics"]

```

Listing 1.9 – Analyse du projet

1.2.10 Scan des Fichiers (_scan_files)

La fonction `_scan_files` a pour rôle d'examiner le dossier donné afin d'y repérer les fichiers Python d'implémentation et de test, et de signaler les éventuelles erreurs détectées durant cette phase.

- **Initialisation du résultat** : Création d'un dictionnaire `result` contenant deux clés principales : `files_found` (initialisé avec `None`) et `errors` (liste vide).
- **Vérification du dossier** : Si le dossier n'existe pas, un message d'erreur est ajouté à `errors`, et le dictionnaire est retourné immédiatement.
- **Détection de l'implémentation** : Recherche du premier fichier Python ne correspondant pas à un fichier de test, et enregistre son chemin comme fichier d'implémentation.
- **Détection des fichiers de test** : Liste tous les fichiers Python considérés comme fichiers de test, et enregistre le premier trouvé.
- **Validation de la détection** : Si aucun fichier d'implémentation n'a été identifié, une erreur est ajoutée à `errors`.
- **Gestion des exceptions** : En cas d'erreur lors de la lecture des fichiers, celle-ci est capturée et ajoutée à la liste `errors`.

Cette fonction constitue la première étape de l'analyse d'un projet : elle identifie les fichiers clés à examiner et prépare le terrain pour les phases d'analyse suivantes.

```

def _scan_files(self, folder: Path) -> Dict:
    result = {"files_found": {"implementation": None, "test": None},
              "errors": []}
    if not folder.exists():
        result["errors"].append(f"Dossier introuvable : {folder}")
    return result

```



```

try:
    for file in folder.glob("*.py"):
        if not self._is_test_file(file):
            result["files_found"]["implementation"] = str(file)
            break

    test_files = [f for f in folder.glob("*.py") if
        self._is_test_file(f)]
    if test_files:
        result["files_found"]["test"] = str(test_files[0])

    if not result["files_found"]["implementation"]:
        result["errors"].append("Aucun fichier d'implémentation")
except Exception as e:
    result["errors"].append(f"Erreur scan fichiers : {e}")

return result

```

Listing 1.10 – Scan des fichiers

1.2.11 Analyse de l'Implémentation (_analyze_implementation)

La fonction `_analyze_implementation` effectue une analyse approfondie d'un fichier Python d'implémentation afin d'évaluer la qualité des fonctions définies, de leurs documentations et de leurs commentaires associés.

- **Initialisation du résultat** : Création d'un dictionnaire contenant des indicateurs de validité syntaxique, le nombre de fonctions détectées, la précision des documentations et des commentaires, ainsi que des listes d'erreurs ou d'éléments manquants.
- **Lecture et analyse du fichier** : Le contenu du fichier est lu, analysé avec l'AST pour vérifier la syntaxe, et les commentaires sont extraits ligne par ligne à l'aide de `tokenize`.
- **Détection des fonctions** : Toutes les fonctions définies au niveau supérieur du fichier sont collectées pour évaluation. Leur nombre est stocké et leurs noms sont affichés dans la console.
- **Vérification des docstrings** :
 - Si une fonction ne contient pas de documentation, son nom est ajouté à `missing_docs`.
 - Sinon, la documentation est nettoyée puis évaluée par le LLM pour vérifier sa pertinence au code.
 - Les résultats (précision, problèmes, justification) sont stockés dans `docstring_issues`.
- **Vérification des commentaires** :
 - Un commentaire situé à proximité de la fonction (sur la même ligne ou lignes suivantes) est recherché.

- Dans le cas d'absence de commentaire, le nom de la fonction est ajouté à `uncommented`.
- Si un commentaire est trouvé, il est nettoyé et évalué par le LLM, puis les résultats sont ajoutés à `comment_issues`.
- **Calcul des scores de précision** : Si au moins une fonction est présente, les ratios de précision des docstrings et des commentaires sont calculés et stockés dans `docstrings_accurate` et `comments_accurate`.
- **Gestion des erreurs** : En cas d'erreur de syntaxe ou d'exception générale lors du traitement, une entrée est ajoutée à `errors`.

Cette fonction constitue le coeur de l'analyse du fichier d'implémentation, combinant parsing syntaxique, extraction sémantique et vérification automatique via LLM pour établir un diagnostic structuré de la qualité documentaire du code.

```
def _analyze_implementation(self, file: Path) -> Dict:
    result = {
        "valid_syntax": 0,
        "function_count": 0,
        "missing_docs": [],
        "uncommented": [],
        "docstrings_accurate": 0.0,
        "docstring_issues": {},
        "comments_accurate": 0.0,
        "comment_issues": {},
        "errors": []
    }

    try:
        with open(file, 'r', encoding='utf-8') as f:
            content = f.read()
            tree = ast.parse(content)
            result["valid_syntax"] = 1

            tokens =
                tokenize.tokenize(BytesIO(content.encode('utf-8')).readline)
            comments_by_line = {t.start[0]: t.string.strip('#').strip()
                                for t in tokens if t.type == tokenize.COMMENT}

            functions = [n for n in tree.body if isinstance(n,
                ast.FunctionDef)]
            result["function_count"] = len(functions)
            print("Fonctions d test es :", [f.name for f in functions])

            accurate_docstrings = 0
```

```

accurate_comments = 0

for func in functions:
    docstring = ast.get_docstring(func)
    if not docstring:
        result["missing_docs"].append(func.name)
    else:
        docstring = self._clean_text(docstring)
        func_code = ast.get_source_segment(content, func)
        if self.llm:
            is_accurate, issues, reason =
                self._verify_docstring(func.name, docstring,
                                       func_code)
            if is_accurate:
                accurate_docstrings += 1
            result["docstring_issues"][func.name] =
                {"issues": issues, "reason": reason,
                "docstring": docstring}

        comment = None
        start_line = func.lineno
        end_line = getattr(func, 'end_lineno', start_line + 10)
        for line in range(start_line, end_line + 1):
            if line in comments_by_line:
                comment = comments_by_line[line]
                break

        print(f"Commentaire pour {func.name} : {comment}")
        if not comment:
            result["uncommented"].append(func.name)
        else:
            comment = self._clean_text(comment)
            func_code = ast.get_source_segment(content, func)
            if self.llm:
                is_accurate, issues, reason =
                    self._verify_comment(func.name, comment,
                                         func_code)
                if is_accurate:
                    accurate_comments += 1
            result["comment_issues"][func.name] =
                {"comment": comment, "issues": issues,
                "reason": reason}

```

```

    if result["function_count"] > 0:
        result["docstrings_accurate"] = accurate_docstrings /
            result["function_count"]
        result["comments_accurate"] = accurate_comments /
            result["function_count"]
    else:
        result["docstrings_accurate"] = 0.0
        result["comments_accurate"] = 0.0

    print(f"Score docstrings_accurate :
          {result['docstrings_accurate']:.2f}
          ({accurate_docstrings}/{result['function_count']})")
    print(f"Score comments_accurate :
          {result['comments_accurate']:.2f}
          ({accurate_comments}/{result['function_count']})")

except SyntaxError:
    result["errors"].append("Erreur syntaxe fichier")
except Exception as e:
    result["errors"].append(f"Erreur analyse : {e}")

return result

```

Listing 1.11 – Analyse de l'implementation

1.2.12 Analyse des Tests (_analyze_tests)

La fonction `_analyze_tests` évalue un fichier de tests Python afin de vérifier la présence, la quantité et la validité des fonctions de test à l'aide de la classe `TestAnalyzer`.

- **Initialisation du résultat** : Création d'un dictionnaire pour stocker les résultats de l'analyse syntaxique, la détection de fonctions de test, leur validité, et les éventuelles erreurs.
- **Initialisation du TestAnalyzer** : Le fichier est transmis à une instance de `TestAnalyzer`, qui exécute une analyse complète via sa méthode `analyze()`.
- **Validation syntaxique** : Si l'analyse ne déclenche pas d'exception, la syntaxe est considérée comme valide (`valid_syntax = 1`).
- **Détection de fonctions de test** :
 - Si le rapport de l'analyseur contient au moins une classe de test et n'indique pas d'échec critique (`NO_TEST_CLASSES`), la présence de tests est confirmée (`has_test_functions = 1`).

- Le nombre total de fonctions de test est extrait à partir des messages du rapport contenant la structure « Classe de test ... trouvée avec ... méthodes ».
- **Évaluation de la validité des tests** : Le score retourné par `analyze()` est utilisé pour déterminer si les tests sont valides, et un résumé est généré avec les problèmes détectés (`failed_tests`).
- **Gestion des erreurs** : Toute exception détectée pendant l'analyse est capturée, ajoutée à `errors`, et un résultat d'échec est stocké dans `test_validity`.

Cette fonction agit comme un intermédiaire entre le fichier de tests Python et le module `TestAnalyzer`, synthétisant les résultats dans une structure exploitable pour le rapport global du projet.

```
def _analyze_tests(self, file: Path) -> Dict:
    print(f"\nUtilisation de TestAnalyzer pour analyser le fichier de
          tests : {file}")
    result = {
        "valid_syntax": 0,
        "has_test_functions": 0,
        "test_function_count": 0,
        "test_validity": {},
        "errors": []
    }

    try:
        test_analyzer = TestAnalyzer(str(file))
        score = test_analyzer.analyze()

        result["valid_syntax"] = 1

        result["has_test_functions"] = 1 if len(test_analyzer.report) >
            0 and "NO_TEST_CLASSES" not in test_analyzer.failed_tests
            else 0

        test_count = 0
        for entry in test_analyzer.report:
            if "Classe de test" in entry and "trouvée avec" in entry:
                try:
                    count_part = entry.split("trouvée
                        avec")[1].split("méthode")[0].strip()
                    test_count += int(count_part)
                except:
                    pass

        result["test_function_count"] = test_count
```

```

    result["test_validity"] = {
        "is_valid": score == 1,
        "issues": test_analyzer.failed_tests,
        "reason": "Tests valides" if score == 1 else "Problèmes
            de tests dans les tests"
    }

    print(f"Analyse des tests complétée. Score : {score}")

except Exception as e:
    result["errors"].append(f"Erreur lors de l'analyse des tests :
        {e}")
    result["test_validity"] = {
        "is_valid": False,
        "issues": [f"Erreur inattendue : {e}"],
        "reason": "Chec de l'analyse du fichier de test"
    }
    print(f"Exception lors de l'analyse des tests : {e}")

return result

```

Listing 1.12 – Analyse des tests

1.2.13 Vérification des Documentations (_verify_docstring)

La fonction `_verify_docstring` vérifie la précision de la documentation en la comparant au code de la fonction concernée à l'aide du modèle de langage (LLM).

- **Vérification de la disponibilité du LLM** : Si aucun modèle n'est disponible, la fonction retourne immédiatement une erreur signalant l'indisponibilité de l'analyse.
- **Nettoyage de la docstring** : Le texte de la documentation est nettoyé via la méthode `_clean_text` afin d'éliminer les caractères indésirables.
- **Détection de la langue** : La langue de la documentation est identifiée automatiquement par la méthode `_detect_language`.
- **Appel au LLM** : La méthode `_call_llm` est invoquée avec les informations nécessaires (nom de la fonction, documentation, code source, langue détectée) ainsi qu'un prompt adapté à l'analyse des documentations.
- **Retour du résultat** : La fonction retourne un triplet contenant : un booléen de validation, une liste des problèmes détectés, et une explication textuelle générée par le LLM.

Cette fonction résume la logique de validation des documentations, en s'appuyant sur un modèle de

langage pour garantir la conformité entre documentation et comportement du code.

```
def _verify_docstring(self, func_name: str, docstring: str, func_code:
    str) -> Tuple[bool, List[str], str]:
    if not self.llm:
        print(f"LLM indisponible pour docstring '{func_name}'")
        return False, ["LLM indisponible"], "LLM indisponible"

    docstring = self._clean_text(docstring)
    lang = self._detect_language(docstring)
    return self._call_llm(
        func_name=func_name,
        prompt_template=self.docstring_prompt,
        content={"func_name": func_name, "docstring": docstring, "code":
            func_code, "lang": lang},
        validation_type="docstring"
    )
```

Listing 1.13 – Verification des documentations

1.2.14 Vérification des Commentaires (_verify_comment)

La fonction `_verify_comment` vérifie la précision d'un commentaire associé à une fonction Python, en le comparant à son code source à l'aide du modèle de langage (LLM).

- **Vérification de la disponibilité du LLM** : Si aucun modèle n'est disponible, un message d'erreur est affiché et la fonction retourne une réponse d'échec.
- **Nettoyage du commentaire** : Le texte du commentaire est nettoyé via la méthode `_clean_text` pour retirer les caractères indésirables.
- **Détection de la langue** : La langue du commentaire est détectée automatiquement à l'aide de la méthode `_detect_language`.
- **Appel au LLM** : Le modèle de langage est invoqué via `_call_llm` avec un prompt spécifique aux commentaires et les données nécessaires : nom de la fonction, commentaire, code source, langue.
- **Retour du résultat** : La fonction retourne un triplet composé d'un booléen indiquant la validité, d'une liste des problèmes identifiés, et d'un texte explicatif généré par le LLM.

Cette fonction assure une validation automatisée et cohérente des commentaires de code à l'aide du modèle de langage (LLM)

```
def _verify_comment(self, func_name: str, comment: str, func_code: str)
    -> Tuple[bool, List[str], str]:
```

```

if not self.llm:
    print(f"LLM indisponible pour commentaire '{func_name}'")
    return False, ["LLM indisponible"], "LLM indisponible"

comment = self._clean_text(comment)
lang = self._detect_language(comment)
return self._call_llm(
    func_name=func_name,
    prompt_template=self.comment_prompt,
    content={"func_name": func_name, "comment": comment, "code":
        func_code, "lang": lang},
    validation_type="comment"
)

```

Listing 1.14 – Vérification des commentaires

1.2.15 Vérification des Tests (_verify_test_file)

La fonction `_verify_test_file` utilise le LLM pour but d'évaluer la validité du contenu d'un fichier de test Python.

- **Vérification de la disponibilité du LLM** : Si aucun LLM n'est configuré, un message d'erreur est affiché et la fonction retourne une erreur.
- **Appel au LLM** : Le contenu du fichier de test est transmis à la méthode `_call_llm`, avec un prompt qui évalue des tests, afin d'analyser sa qualité et sa validité.
- **Retour du résultat** : La fonction retourne un booléen qui indique si les tests sont valides, ainsi qu'une liste des problèmes.

Cette méthode permet une vérification automatique de la qualité des fichiers de test à l'aide du LLM.

```

def _verify_test_file(self, test_code: str) -> Tuple[bool, List[str]]:
    if not self.llm:
        print("LLM indisponible pour validation des tests")
        return False, ["LLM indisponible"]

    return self._call_llm(
        func_name="test_file",
        prompt_template=self.test_prompt,
        content={"test_code": test_code},
        validation_type="test"
    )

```

Listing 1.15 – Vérification des tests unitaires

1.2.16 Appel au LLM (_call_llm)

La fonction `_call_llm` envoie un prompt formaté au LLM afin de valider automatiquement les documentations, commentaires ou fichiers de test, et interprète la réponse retournée.

- **Initialisation de la boucle de tentatives** : Trois essais sont autorisés en cas d'échec, avec un délai de 60 secondes d'attente entre deux tentatives.
- **Envoi du prompt** : Le prompt est généré à partir d'un template, injecté avec les valeurs spécifiques, puis transmis au LLM à l'aide de `self.llm.invoke`.
- **Sauvegarde et nettoyage de la réponse** : La réponse brute est enregistrée sur disque, puis nettoyée des balises Markdown pour extraire une chaîne JSON exploitable. (Markdown est un langage de balisage qui permet de structurer des contenus comme les titres, les listes numérotés ou à puces)
- **Vérification des clés** : Selon le type de validation (`docstring`, `comment`, ou `test`), la présence des clés attendues est contrôlée. Toute anomalie inattendue est rapportée et la réponse est sauvegardée comme échec.
- **Retour structuré** : En fonction du type de validation, la fonction retourne des éléments qui indiquent la validité, les problèmes détectés et la justification fournie par le LLM.

Cette fonction est au coeur de la validation par le LLM, garantissant robustesse, traçabilité, et interprétation fiable des réponses générées par le modèle.

```
def _call_llm(self, func_name: str, prompt_template: PromptTemplate,
             content: Dict, validation_type: str) -> Tuple:
    max_retries = 3
    retry_delay = 60

    for attempt in range(max_retries):
        try:
            prompt = prompt_template.format(**content)
            response = self.llm.invoke(prompt)
            response_text = response.content.strip()

            print(f"R ponse brute LLM pour {validation_type} de\n"
                  f"{func_name}:\n{response_text}\n")
            with open(self.report_dir /
                      f"llm_response_{func_name}_{validation_type}.txt", 'w',
                      encoding='utf-8') as f:
                f.write(response_text)

            cleaned_response = re.sub(r'```json\n|```$', '',
                                     response_text, flags=re.MULTILINE).strip()
```

```

cleaned_response = re.sub(r'^```|```$', '',
    cleaned_response, flags=re.MULTILINE).strip()

print(f"R ponse nettoy e LLM pour {validation_type} de
    {func_name}: {cleaned_response}")
try:
    result = json.loads(cleaned_response)
except json.JSONDecodeError:
    if match := re.search(r'\{[\s\S]*?\}', cleaned_response,
        re.DOTALL):
        result = json.loads(match.group(0))
    else:
        print(f"JSON invalide pour {validation_type} de
            '{func_name}'")
        self._save_failed_response(func_name, response_text)
        return (False, [f"JSON invalide - {validation_type}
            non valide"], "JSON invalide") if validation_type
            in ["docstring", "comment"] else (False, [f"JSON
            invalide"])

if not isinstance(result, dict):
    print(f"Structure JSON incorrecte pour {validation_type}
        de '{func_name}'")
    self._save_failed_response(func_name, response_text)
    return (False, [f"Structure JSON incorrecte"],
        "Structure JSON incorrecte") if validation_type in
        ["docstring", "comment"] else (False, [f"Structure
        JSON incorrecte"])

if validation_type == "test":
    required_keys = {"is_valid", "issues", "reason"}
    missing_keys = required_keys - result.keys()
    if missing_keys:
        print(f"C l s manquantes pour {validation_type} de
            '{func_name}' : {missing_keys}")
        self._save_failed_response(func_name, response_text)
        return False, [f"C l s manquantes : {'
            '.join(missing_keys)}"]
    is_valid = bool(result.get("is_valid", False))
    issues = result.get("issues", []) if
        isinstance(result.get("issues"), list) else []
    return is_valid, issues
else:

```

```

required_keys = {"is_accurate", "issues", "reason"}
missing_keys = required_keys - result.keys()
if missing_keys:
    print(f"Clés manquantes pour {validation_type} de  

        '{func_name}' : {missing_keys}")
    self._save_failed_response(func_name, response_text)
    return False, [f"Clés manquantes : {'  

        '.join(missing_keys)}"], f"Clés manquantes : {'  

        '.join(missing_keys)}"
    return (
        bool(result.get("is_accurate", False)),
        result.get("issues", []) if
            isinstance(result.get("issues"), list) else [],
        result.get("reason", "")
    )

except google.api_core.exceptions.ResourceExhausted as e:
    print(f"Limite requêtes pour {validation_type} de  

        '{func_name}' : {e}")
    if attempt < max_retries - 1:
        print(f"Attente {retry_delay}s...")
        time.sleep(retry_delay)
        continue
    print(f"Chec après {max_retries} tentatives")
    return (False, [f"Limite requêtes dépassée : {e}"],
        "Limite requêtes dépassée") if validation_type in
        ["docstring", "comment"] else (False, [f"Limite requêtes  

        dépassée : {e}"])

except Exception as e:
    print(f"Erreur LLM pour {validation_type} de '{func_name}' :  

        {e}")
    self._save_failed_response(func_name, response_text if
        'response_text' in locals() else "Aucune réponse")
    return (False, [f"Erreur LLM : {e}"], "Erreur LLM") if
        validation_type in ["docstring", "comment"] else (False,
        [f"Erreur LLM : {e}"])

```

Listing 1.16 – Appeler le LLM

1.2.17 Sauvegarde Réponses Échouées (_save_failed_response)

La fonction `_save_failed_response` enregistre les réponses incorrectes ou invalides retournées par le LLM, afin de faciliter le débogage et la traçabilité.

- **Définition du chemin de sauvegarde** : Génère un chemin de fichier à partir du nom de la fonction, dans le répertoire de rapport configuré.
- **Écriture sur disque** : Ouvre le fichier en écriture avec encodage UTF-8 et y écrit le nom de la fonction ainsi que le contenu brut de la réponse.
- **Confirmation par message console** : Affiche un message indiquant que la réponse a bien été sauvegardée.
- **Gestion des erreurs** : Capture toute exception liée à l'écriture du fichier et affiche un message d'échec détaillé.

Cette fonction assure la conservation des réponses mal formatées, facilitant les vérifications à faire et l'amélioration de la fiabilité du système.

```
def _save_failed_response(self, func_name: str, response_text: str) ->
    None:
    failed_path = self.report_dir / f"failed_response_{func_name}.txt"
    try:
        with open(failed_path, 'w', encoding='utf-8') as f:
            f.write(f"Fonction : {func_name}\nR ponse
                    :\n{response_text}")
        print(f"R ponse   choue   sauvegard e : {failed_path}")
    except Exception as e:
        print(f"  chec   sauvegarde r ponse '{func_name}' : {e}")
```

Listing 1.17 – Sauvgardage des réponses échoués

1.2.18 Calcul des Métriques (_compute_metrics)

La fonction `_compute_metrics` extrait des indicateurs à partir du rapport généré par les différentes phases d'analyse, afin de produire une vue globale du projet audité.

- **Initialisation des métriques** : Copie une structure vide depuis `metrics_template` pour y stocker les résultats.
- **Analyse des fichiers détectés** : Récupère les données des phases 1 et 2 du rapport, notamment la présence de fichiers d'implémentation et de tests.
- **Tests unitaires** : Détermine si des tests existent et s'ils sont valides en vérifiant les résultats d'analyse de la phase 2.

- **Analyse de l'implémentation** : Si un fichier d'implémentation est présent, les métriques suivantes sont évaluées : syntaxe valide, présence de toutes les documentations, commentaires, et leurs précision .
- **Retour du résumé** : Renvoie le dictionnaire `metrics` contenant l'ensemble des indicateurs calculés.

Cette fonction centralise l'interprétation des résultats bruts en un ensemble cohérent de métriques utiles pour l'évaluation automatique de la qualité du code.

```
def _compute_metrics(self, report: Dict) -> Dict:
    metrics = self.metrics_template.copy()
    phase1 = report["phase1"]
    phase2 = report["phase2"]

    metrics["files_exist"] = 1 if
        phase1["files_found"].get("implementation") else 0
    test_analysis = phase2.get("test_analysis", {})
    metrics["has_tests"] = 1 if phase1["files_found"].get("test") else 0
    metrics["tests_valid"] = 1 if test_analysis.get("test_validity",
        {}).get("is_valid", False) else 0

    if metrics["files_exist"]:
        impl = phase2["implementation_analysis"]
        metrics["files_valid"] = impl["valid_syntax"]
        metrics["fully_documented"] = 0 if impl["missing_docs"] else 1
        metrics["all_functions_commented"] = 0 if impl["uncommented"]
            else 1
        metrics["docstrings_accurate"] = impl["docstrings_accurate"]
        metrics["comments_accurate"] = impl["comments_accurate"]

    return metrics
```

Listing 1.18 – Calcul des métriques

1.2.19 Rendu Rapport Terminal (`_render_terminal_report`)

La fonction `_render_terminal_report` génère un rapport textuel destiné à l'affichage en terminal, résumant les résultats des deux phases d'analyse (structure et qualité du code).

- **Section Structure Fichiers** :
 - Affiche la présence des fichiers d'implémentation et de tests.

- Si un fichier de tests est trouvé, il évalue sa syntaxe, le nombre de fonctions de test détectées, et leur validité.
- Liste les éventuelles erreurs détectées lors de la phase 1.
- **Section Qualité Code :**
 - Présente des statistiques sur le nombre de fonctions, la validité des syntaxes, les fonctions non documentées ou non commentées.
 - Affiche des scores de précision pour les documentations et commentaires.
 - Énumère les fonctions sans commentaires ou sans documentations si elles existent.
 - Liste les problèmes détectés dans les documentations ou les commentaires avec détails.
- **Section Métriques Finales :**
 - Récapitule les métriques globales calculées (présence des fichiers, validité des tests, précision de la documentation, etc.) dans un tableau lisible.
- **Retour :** Renvoie une chaîne de caractères multi-lignes contenant l'intégralité du rapport, prête à être affichée en terminal.

```
def _render_terminal_report(self, report: Dict) -> str:
    lines = ["=== RAPPORT FINAL ANALYSE CODE ===", "\n### Structure\nFichiers"]
    phase1 = report["phase1"]
    phase2 = report["phase2"]

    lines.append(f"Impl mentation :\n{phase1['files_found'].get('implementation', 'Introuvable')}")
    if test_file := phase1["files_found"].get("test"):
        test_analysis = phase2.get("test_analysis", {})
        if not test_analysis.get("valid_syntax", 0):
            lines.append(f"Fichier Tests : {test_file} (syntaxe\ninvalide)")
        elif test_analysis.get("has_test_functions", 0):
            lines.append(f"Fichier Tests : {test_file}\n({test_analysis['test_function_count']} fonctions de\ntest)")
            test_validity = test_analysis.get("test_validity", {})
            lines.append(f"Tests Valides : {'Oui' if\ntest_validity.get('is_valid', False) else 'Non'}")
            if issues := test_validity.get("issues", []):
                lines.append("***Probl mes Tests :**")
                lines.extend(f"- {issue}" for issue in issues)
        else:
            lines.append(f"Fichier Tests : {test_file} (aucune fonction\nde test)")
```

```

else:
    lines.append("Fichier Tests : Introuvable")

lines.extend([f"Erreur : {e}" for e in phase1["errors"]])
lines.append("\n### Qualit Code")

impl = phase2["implementation_analysis"]
if impl:
    uncommented_count = len(impl["uncommented"])
    missing_docs_count = len(impl["missing_docs"])
    total_functions = impl["function_count"]
    lines.extend([
        f"\nFonctions : {total_functions}",
        f"Syntaxe Valide : {'Oui' if impl['valid_syntax'] else 'Non'}",
        f"Fonctions Sans Docstring : {missing_docs_count}/{total_functions}",
        f"Fonctions Sans Commentaire : {uncommented_count}/{total_functions}",
        f"Docstrings Pr cises : {impl['docstrings_accurate']:.2f}",
        f"Commentaires Pr cis : {impl['comments_accurate']:.2f}",
    ])

    if impl["uncommented"]:
        lines.append("\n**Fonctions Non Comment es :**")
        lines.extend(f"- {name}" for name in impl["uncommented"])

    if impl["missing_docs"]:
        lines.append("\n**Fonctions Non Document es :**")
        lines.extend(f"- {name}" for name in impl["missing_docs"])

    if doc_issues := phase2.get("docstring_issues", {}):
        lines.append("\n**Probl mes Docstrings :**")
        for func_name, data in doc_issues.items():
            lines.append(f"- {func_name} (Docstring : \"{data['docstring']}\") :")
            lines.append(f" Raison : {data['reason']}")
            if data["issues"]:
                lines.extend(f" - {issue}" for issue in data["issues"])

    if com_issues := phase2.get("comment_issues", {}):
        lines.append("\n**Probl mes Commentaires :**")

```

```

        for func_name, data in com_issues.items():
            lines.append(f"- {func_name} (Commentaire :
                           \"{data['comment']}\") :")
            lines.append(f"  Raison : {data['reason']}")
            if data["issues"]:
                lines.extend(f"    - {issue}" for issue in
                             data["issues"])

    lines.extend([
        "\n### Métriques Finales",
        "| Métrique                | Valeur |",
        "|-----|-----|",
        *[f"| {k.replace('_', ' ').title():19}| {v:.2f} |" if
          isinstance(v, float) else f"| {k.replace('_', '
          ').title():19}| {v} |"
          for k, v in report["combined_metrics"].items()
    ])

    return "\n".join(lines)

```

Listing 1.19 – Rapport Terminal

1.2.20 Sauvegarde du Rapport (_save_report)

La fonction `_save_report` sauvegarde les métriques combinées de l'analyse d'un projet dans un fichier JSON nommé à partir du dossier cible.

— **Paramètres :**

- `folder_path` (str) : chemin vers le dossier du projet analysé.
- `report` (Dict) : dictionnaire contenant les résultats complets de l'analyse, notamment la clé `combined_metrics`.

— **Comportement :**

- Génère un nom de fichier de rapport à partir du nom du dossier cible.
- Extrait les métriques principales depuis `combined_metrics` dans un ordre défini (structure, tests, documentation, précision).
- Tente d'écrire ces métriques dans un fichier JSON dans le répertoire de rapports.
- Si une erreur de permission est détectée (Cas de permission nécessaire pour ajouter un fichier dans la répertoire indiquée), tente une sauvegarde de secours dans le répertoire courant.
- En cas d'échec complet, affiche un message d'erreur approprié.

— Fichier généré :

- `report_{nom_du_dossier}.json` : contient les métriques sous forme de dictionnaire ordonné pour but de faciliter sa lisibilité.

```
def _save_report(self, folder_path: str, report: Dict) -> None:
    folder_name = Path(folder_path).name
    json_file = self.report_dir / f"report_{folder_name}.json"
    metrics = report["combined_metrics"]

    ordered_metrics = {
        "files_exist": metrics["files_exist"],
        "files_valid": metrics["files_valid"],
        "has_tests": metrics["has_tests"],
        "tests_valid": metrics["tests_valid"],
        "fully_documented": metrics["fully_documented"],
        "all_functions_commented": metrics["all_functions_commented"],
        "docstrings_accurate": metrics["docstrings_accurate"],
        "comments_accurate": metrics["comments_accurate"]
    }

    print(f"Sauvegarde rapport : {json_file}")
    try:
        with open(json_file, 'w', encoding='utf-8') as f:
            json.dump(ordered_metrics, f, indent=4)
        print(f"Rapport sauvegardé : {json_file}")
    except PermissionError as e:
        print(f"Permission refusée : {json_file}: {e}")
        fallback = Path.cwd() / f"report_{folder_name}.json"
        try:
            with open(fallback, 'w', encoding='utf-8') as f:
                json.dump(ordered_metrics, f, indent=4)
            print(f"Rapport sauvegardé (secours) : {fallback}")
        except Exception as e2:
            print(f"Chec sauvegarde secours : {e2}")
    except Exception as e:
        print(f"Chec sauvegarde : {json_file}: {e}")
```

Listing 1.20 – Sauvegarde des rapports sous forme .JSON

1.2.21 Vérification Fichier Test (`_is_test_file`)

La fonction `_is_test_file` détermine si un fichier donné est probablement un fichier de test en se basant sur son nom.

— Paramètres :

— path (Path) : chemin vers le fichier à analyser.

— Retour :

— bool : True si le nom du fichier commence par test_ ou contient test , sinon False.

```
def _is_test_file(self, path: Path) -> bool:
    return path.name.startswith('test_') or 'test' in path.name.lower()
```

Listing 1.21 – Vérification du fichier test

1.2.22 Bloc Principal (__main__)

Le bloc principal du script permet d'exécuter l'agent d'analyse de code Python en mode interactif depuis le terminal.

— Comportement :

- Crée une instance de la classe CodeAnalysisAgent.
- Affiche un en-tête signalant le démarrage de l'analyseur.
- Lance une boucle interactive dans laquelle l'utilisateur peut entrer un chemin de dossier à analyser.
- Si l'utilisateur entre q, la boucle se termine et le programme s'arrête.
- Pour chaque dossier saisi, l'agent lance l'analyse du projet avec analyze_project, puis affiche les métriques finales sous forme JSON.

```
if __name__ == "__main__":
    agent = CodeAnalysisAgent()
    print("=== ANALYSEUR CODE PYTHON ===")

    while True:
        path = input("\nChemin dossier projet (ou 'q' pour quitter) : ")
        path = path.strip()
        if path.lower() == 'q':
            break
        metrics = agent.analyze_project(path)
        print(f"\nMétriques retournées : {json.dumps(metrics, indent=4)}")
```

Listing 1.22 – Programme Principal

1.3 Analyse de la Classe TestAnalyzer

1.3.1 Importation des bibliothèques

Les bibliothèques suivantes sont importées pour le bon fonctionnement de l'agent d'analyse de code basé sur une interface graphique :

- `ast` : Fournit des outils pour analyser la structure syntaxique du code Python sous forme d'arbre (Abstract Syntax Tree).
- `os` : Permet de manipuler le système de fichiers (par exemple, naviguer dans les répertoires).
- `tkinter`, `filedialog`, `messagebox` : Permettent de construire une interface graphique avec des fenêtres de dialogue pour sélectionner des dossiers ou afficher des messages.
- `langchain_google_genai.ChatGoogleGenerativeAI` : Fournit une interface pour interagir avec le modèle Gemini de Google via LangChain.
- `google.api_core.exceptions.ResourceExhausted` : Exception utilisée pour intercepter les erreurs de quota dépassé lors des appels à l'API Google.
- `dotenv.load_dotenv` : Charge automatiquement les variables d'environnement définies dans le fichier `.env`, notamment pour l'authentification à l'API Gemini.
- `typing.Optional` : Permet d'annoter les types optionnels (valeurs pouvant être `None`).
- `time` : Utilisé pour gérer les délais d'attente (ex. : gestion des délais entre deux appels à l'API).

```
import ast
import os
import tkinter as tk
from tkinter import filedialog, messagebox
from langchain_google_genai import ChatGoogleGenerativeAI
from google.api_core.exceptions import ResourceExhausted
from dotenv import load_dotenv
from typing import Optional
import time
```

Listing 1.23 – Importation des bibliothèques

1.3.2 Initialisation de l'Analyseur (`__init__`)

Cette méthode initialise une instance de l'agent d'analyse de fichiers de test.

- `test_file_path` : Chemin du fichier de tests à analyser. Il est stocké comme attribut d'instance.
- `self.report` : Liste vide destinée à recevoir les rapports d'analyse LLM.

- `self.failed_tests` : Liste vide utilisée pour enregistrer les tests jugés non conformes ou erronés.
- Un message est affiché dans la console pour indiquer le début de l'analyse du fichier de test.
- `self.llm` : Initialisation de l'agent LLM via la méthode interne `_init_llm`, nécessaire à la validation sémantique des tests.

```
def __init__(self, test_file_path):
    self.test_file_path = test_file_path
    self.report = []
    self.failed_tests = []
    print(f"\n--- Initialisation de l'analyse pour {test_file_path} ---")
    self.llm = self._init_llm()
```

Listing 1.24 – Initialisation de l'Analyseur

1.3.3 Initialisation du LLM (`_init_llm`)

Cette méthode initialise le modèle LLM de Google utilisé pour l'analyse des tests.

- Récupère la clé d'API à partir de la variable d'environnement `GOOGLE_API_KEY`.
- Si la clé est absente, une erreur est levée avec un message explicite.
- Instancie un objet `ChatGoogleGenerativeAI` avec les paramètres suivants :
 - `model` : `gemini-2.0-flash-lite`
 - `temperature` : `0.3` (pour des réponses plus déterministes)
 - `google_api_key` : clé d'API fournie
- En cas de succès, retourne l'objet LLM prêt à être utilisé.
- En cas d'échec, un message d'erreur est affiché et la méthode retourne `None`.

```
def _init_llm(self) -> Optional[ChatGoogleGenerativeAI]:
    print(" tape : Initialisation du LLM...")
    try:
        api_key = os.getenv("GOOGLE_API_KEY")
        if not api_key:
            print("AVERTISSEMENT: GOOGLE_API_KEY manquant")
            raise ValueError("GOOGLE_API_KEY manquant")
        llm = ChatGoogleGenerativeAI(
            model="gemini-2.0-flash-lite",
            google_api_key=api_key,
            temperature=0.3
        )
        print("LLM initialis (Gemini 2.0 Flash Lite)")
```

```

    return llm
except Exception as e:
    print(f"ERREUR:  chec  initialisation LLM: {e}")
    return None

```

Listing 1.25 – Initialisation du LLM

1.3.4 Analyse Statique Tests (parse_test_file)

Cette méthode effectue une analyse statique du fichier de test Python spécifié lors de l'instanciation de l'agent.

— **Validation du chemin :**

- Vérifie que le fichier existe.
- Si le fichier est introuvable, un message d'erreur est ajouté au rapport et la méthode retourne None.

— **Analyse du fichier :**

- Tente d'ouvrir et analyser le fichier avec le module ast.
- En cas d'erreur de syntaxe, l'erreur est consignée, et le tag SYNTAX_ERROR est ajouté à la liste des échecs.
- En cas d'échec d'ouverture ou autre exception, le tag FILE_READ_ERROR est ajouté.

— **Extraction des classes et méthodes de test :**

- Parcourt l'arbre syntaxique pour identifier les classes héritant de TestCase.
- Pour chaque classe valide, extrait les méthodes dont le nom commence par test_.
- Enregistre les noms et le code source (avec ast.unparse) des méthodes.
- Enregistre les classes trouvées dans le rapport avec leur nombre de méthodes.
- Si aucune classe de test n'est trouvée, ajoute le tag NO_TEST_CLASSES.
- Si une classe ne contient aucune méthode, ajoute un avertissement et un tag

— **Retour :**

- Retourne un tuple contenant :
 - L'arbre AST du fichier.
 - Un dictionnaire contenant le code des méthodes de test.

```

def parse_test_file(self):
    print("\n tape: Analyse statique de la structure du fichier...")
    if not os.path.exists(self.test_file_path):
        print(f"ERREUR: Le fichier {self.test_file_path} n'existe pas")
        self.report.append(f"Erreur: Le fichier {self.test_file_path}
                           n'existe pas")

```

```

    return None, {}

try:
    with open(self.test_file_path, 'r', encoding='utf-8') as file:
        file_content = file.read()
        tree = ast.parse(file_content, filename=self.test_file_path)
        print("Fichier pars avec succès")
except SyntaxError as e:
    print(f"ERREUR: Erreur de syntaxe dans {self.test_file_path}: {e}")
    self.report.append(f"Erreur de syntaxe dans {self.test_file_path}: {e}")
    self.failed_tests.append("SYNTAX_ERROR")
    return None, {}
except Exception as e:
    print(f"ERREUR: Erreur lors de la lecture du fichier {self.test_file_path}: {e}")
    self.report.append(f"Erreur lors de la lecture du fichier {self.test_file_path}: {e}")
    self.failed_tests.append("FILE_READ_ERROR")
    return None, {}

test_classes = []
test_methods_code = {}
print("Recherche des classes de test...")
for node in ast.walk(tree):
    if isinstance(node, ast.ClassDef):
        inherits_testcase = any(
            isinstance(base, ast.Name) and base.id == 'TestCase'
            for base in node.bases
        ) or any(
            isinstance(base, ast.Attribute) and base.attr == 'TestCase'
            for base in node.bases
        )
        if inherits_testcase:
            test_methods = []
            for method in node.body:
                if isinstance(method, ast.FunctionDef) and method.name.startswith('test_'):
                    test_methods.append(method.name)
                    method_code = ast.unparse(method)
                    test_methods_code[method.name] = method_code

```

```

        print(f" - M thode trouv e: {method.name}")
        test_classes.append({'name': node.name, 'methods':
            test_methods})
        print(f"Classe de test trouv e: {node.name} avec
            {len(test_methods)} m thode(s)")

    if not test_classes:
        print(f"AVERTISSEMENT: Aucune classe de test valide trouv e
            dans {self.test_file_path}")
        self.report.append(f"Aucune classe de test valide trouv e dans
            {self.test_file_path}")
        self.failed_tests.append("NO_TEST_CLASSES")
    else:
        for cls in test_classes:
            self.report.append(f"Classe de test {cls['name']} trouv e
                avec {len(cls['methods'])} m thode(s) de test")
            if not cls['methods']:
                print(f"AVERTISSEMENT: {cls['name']} n'a aucune m thode
                    de test")
                self.report.append(f"Avertissement: {cls['name']} n'a
                    aucune m thode de test")
                self.failed_tests.append(f"{cls['name']}_NO_METHODS")

    return tree, test_methods_code

```

Listing 1.26 – Analyse des tests

1.3.5 Vérification des Assertions (check_assertions)

Cette méthode vérifie la présence d'assertions dans les fonctions de test d'un fichier analysé, à partir de son arbre syntaxique (AST).

— **Paramètre :**

— `tree` : L'arbre syntaxique AST du fichier de test (généré par `ast.parse`).

— **Pré-condition :**

— Si `tree` est `None`, la méthode arrête l'exécution et affiche une erreur.

— **Traitement :**

— Parcourt toutes les fonctions dont le nom commence par `test_`.

— Pour chaque fonction :

— Recherche la présence d'instructions d'assertion :

- Appels de méthodes
- Instructions Python natives.
- Si aucune assertion n'est trouvée, un avertissement est consigné dans le rapport.
- **Comportement global :**
 - Si aucune assertion n'est détectée dans tout le fichier, un avertissement global est ajouté au rapport.
 - Sinon, un message indique que des assertions ont été trouvées.

```
def check_assertions(self, tree):
    print("\n tape : V rification des assertions...")
    if not tree:
        print("ERREUR: Impossible de v rifier les assertions (arbre AST
              non disponible)")
        return

    assertion_found = False
    for node in ast.walk(tree):
        if isinstance(node, ast.FunctionDef) and
            node.name.startswith('test_'):
            has_assert = False
            for n in ast.walk(node):
                if isinstance(n, ast.Call):
                    if isinstance(n.func, ast.Attribute) and
                        n.func.attr.startswith('assert'):
                        has_assert = True
                        assertion_found = True
                elif isinstance(n, ast.Assert):
                    has_assert = True
                    assertion_found = True

            if has_assert:
                print(f"{node.name}: Assertions trouv es")
            else:
                print(f"AVERTISSEMENT: {node.name}: Aucune assertion
                      trouv e")
                self.report.append(f"Avertissement: La m thode
                                   {node.name} n'a pas d'assertion")

    if not assertion_found:
        print("AVERTISSEMENT: Aucune assertion trouv e dans les tests")
        self.report.append("Avertissement: Aucune assertion trouv e
                           dans les tests")
```



```

else:
    print("Des assertions ont été trouvées dans certains tests")

```

Listing 1.27 – Vérification des assertions

1.3.6 Analyse Sémantique Tests (analyze_test_semantics)

Cette méthode effectue une évaluation sémantique des méthodes de test unitaire à l'aide du LLM. Elle vérifie la qualité, la cohérence et la validité logique des tests. En cas d'échec, elle produit un rapport contenant des explications et des suggestions d'amélioration.

```

def analyze_test_semantics(self, test_methods_code):
    print("\n type: Analyse sémantique avec LLM...")
    if not self.llm:
        print("AVERTISSEMENT: LLM non disponible pour l'analyse sémantique")
        self.report.append("Avertissement: LLM non disponible pour l'analyse sémantique")
        return

    if not test_methods_code:
        print("AVERTISSEMENT: Aucune méthode de test à analyser")
        return

    print(f"Analyse de {len(test_methods_code)} méthodes de test...")
    for method_name, code in test_methods_code.items():
        print(f" - Analyse de {method_name}...")
        prompt = f"""Analyse le test unitaire suivant en Python avec des critères stricts. Un test est considéré VALIDE uniquement s'il satisfait TOUS les critères suivants :
1. Il contient au moins une assertion pertinente (par exemple, 'assertEqual', 'assertTrue', 'assertRaises', etc.) qui vérifie correctement la logique attendue.
2. La logique du test est correcte et cohérente, sans erreurs potentielles d'exécution (par exemple, variables non définies, appels des méthodes inexistantes).
3. Les assertions sont spécifiques et testent des comportements précis (par exemple, éviter les assertions génériques comme 'assertTrue(True)').
4. Le test suit les bonnes pratiques des tests unitaires (par exemple, clarté, indépendance, absence de dépendances externes non gérées).

```

```

[... Contenu du prompt abrégé pour lisibilité ...]"""
    max_retries = 3
    for attempt in range(max_retries):
        try:
            print(f"Envoi de la requête au LLM (tentative {attempt}
                  + 1}/{max_retries})...")
            response = self.llm.invoke(prompt)
            analysis = response.content if hasattr(response,
            'content') else str(response)

            is_valid = "Valide" in analysis and "Non valide" not in
            analysis
            score = 1 if is_valid else 0

            if not is_valid:
                self.failed_tests.append(method_name)
                print(f"      {method_name}: Considéré non valide
                      par le LLM")
            else:
                print(f"      {method_name}: Considéré valide par le
                      LLM")

            lines = analysis.split("\n")
            explanation = ""
            suggestions = ""
            current_section = None
            for line in lines:
                if line.startswith("Explication :"):
                    current_section = "explanation"
                    explanation = line[len("Explication :"):].strip()
                elif line.startswith("Suggestions :"):
                    current_section = "suggestions"
                    suggestions = line[len("Suggestions :"):].strip()
                elif current_section == "explanation":
                    explanation += " " + line.strip()
                elif current_section == "suggestions":
                    suggestions += " " + line.strip()

            summary = explanation.replace("\n", " ").strip()
            if len(summary) > 150:
                summary = summary[:147] + "..."

            self.report.append(f"Analyse LLM pour {method_name}:

```

```

        {'Valide' if is_valid else 'Non valide'} (Score:
        {score}))")
    self.report.append(f"  Explication: {summary}")
    if suggestions and not is_valid:
        self.report.append(f"  Suggestions: {suggestions}")

    break

except ResourceExhausted as e:
    if attempt < max_retries - 1:
        print(f"    Limite de requêtes atteinte. Attente de
              60 secondes avant de réessayer...")
        time.sleep(60)
    else:
        print(f"    échec après {max_retries} tentatives
              pour {method_name}. Attribution de score 0.")
        self.report.append(f"Erreur: Limite de requêtes
                          d'essai pour {method_name} après
                          {max_retries} tentatives")
        self.failed_tests.append(method_name)
        break

except Exception as e:
    print(f"    ERREUR: Erreur inattendue lors de l'analyse
          LLM pour {method_name}: {e}")
    self.report.append(f"Erreur inattendue lors de l'analyse
                      LLM pour {method_name}: {e}")
    self.failed_tests.append(method_name)
    break

```

Explication du Prompt

- Le prompt fourni au LLM est structuré pour imposer une évaluation rigoureuse et binaire (valide / non valide) des tests unitaires.
- **Critères d'évaluation :**
 1. Le test doit contenir au moins une assertion pertinente
 2. Le corps du test ne doit contenir aucune erreur de logique ou de syntaxe évidente, comme l'utilisation de variables non définies ou de méthodes inexistantes.
 3. Les assertions doivent être spécifiques et viser un comportement clair, en évitant les assertions triviales ou inutiles.
 4. Le test doit respecter les bonnes pratiques des tests unitaires : clarté, indépendance, lisibilité, et absence de dépendances externes incontrôlées.

- Le prompt précise que **tous les critères doivent être satisfaits simultanément** pour que le test soit considéré comme “Valide”, ce qui impose une exigence stricte et évite les jugements ambigus.
- La sortie attendue du LLM doit contenir explicitement les sections suivantes :
 - Valide ou Non valide (avec cette formulation exacte pour un traitement automatisé).
 - Une section “**Explication :**” résumant les raisons de la décision.
 - Une section “**Suggestions :**” optionnelle, mais fortement encouragée en cas de test non valide, fournissant des pistes concrètes d’amélioration.
- En cas d’erreur ou de saturation API (ex. `ResourceExhausted`), un mécanisme de relance avec temporisation est mis en place, jusqu’à 3 tentatives par méthode avec 60 secondes entre deux tentatives (`time.sleep(60)`)

1.3.7 Analyse Complète Tests (analyze)

Cette méthode orchestre l’ensemble du processus d’analyse du fichier de test. Elle combine l’analyse statique de la structure du code, la vérification des assertions et une validation sémantique via LLM. Un score final est calculé en fonction des échecs détectés et un rapport détaillé est généré.

Logique de traitement étape par étape :

1. Affiche une bannière de début d’analyse pour indiquer clairement l’identité du fichier traité.
2. Appelle `parse_test_file()` pour extraire l’arbre AST et les méthodes de test avec leur code source.
3. Lance `check_assertions(tree)` pour inspecter chaque méthode de test et détecter la présence d’assertions.
4. Si des méthodes de test sont détectées, déclenche l’analyse sémantique via `analyze_test_semantics()`, qui interroge le LLM pour évaluer la pertinence logique de chaque test.
5. Détermine ensuite si des tests ont échoué (mauvaise structure, absence d’assertions, invalidité sémantique) :
 - Si oui : score de 0, et les noms des méthodes fautives sont listés dans le rapport.
 - Sinon : score de 1, indiquant que tous les tests sont jugés valides.
6. Le rapport est affiché ligne par ligne dans la console, incluant les messages d’alerte et d’analyse accumulés.
7. Affiche une bannière de fin d’analyse et retourne le score global pour intégration ou évaluation.

```
def analyze(self):
    print("\n===== ")
    print(f"D BUT DE L'ANALYSE: {self.test_file_path}")
```

```

print("=====")
self.report.append(f"Analyse du fichier: {self.test_file_path}")

tree, test_methods_code = self.parse_test_file()
self.check_assertions(tree)

if test_methods_code:
    self.analyze_test_semantics(test_methods_code)
else:
    print("AVERTISSEMENT: Analyse s mantique ignor e (aucune
          m thode de test trouv e)")

print("\n=====")
print("R SUM DE L'ANALYSE")
print("=====")

has_failed_tests = len(self.failed_tests) > 0

if has_failed_tests:
    score = 0
    print(f"Score: {score} (Tests non valides)")

    failed_set = set(self.failed_tests)
    if failed_set:
        failed_list = ", ".join(failed_set)
        print(f"M thodes de test erron es: {failed_list}")
        self.report.append(f"M thodes de test erron es: " +
                           failed_list)

    self.report.append(f"Score: {score} (Tests non valides -
                        incoh rences d tect es)")
else:
    score = 1
    print(f"Score: {score} (Tests valides - pas d'incoh rences
          d tect es)")
    self.report.append(f"Score: {score} (Tests valides)")

print("\n=====")
print("RAPPORT D'ANALYSE COMPLET")
print("=====")
for entry in self.report:
    print(f"- {entry}")

```

```

print ("\n===== ")
print ("FIN DE L'ANALYSE")
print ("===== ")

return score

```

Listing 1.28 – L'Analyse complète des tests

1.3.8 Sélection Fichier Analyse (select_and_analyze)

Cette fonction fournit une interface graphique minimaliste permettant à l'utilisateur de sélectionner un fichier de test Python à analyser. Elle utilise Tkinter pour afficher une boîte de dialogue, et appelle ensuite l'analyseur de test sur le fichier sélectionné. Si aucun fichier n'est choisi, une alerte est affichée.

Logique de traitement étape par étape :

1. Initialise une instance Tkinter cachée (`root.withdraw()`) pour permettre une interaction graphique sans fenêtre principale.
2. Ouvre une boîte de dialogue via `filedialog.askopenfilename` pour que l'utilisateur sélectionne un fichier Python.
3. Si l'utilisateur annule la sélection :
 - Affiche une boîte d'alerte avec `messagebox.showwarning`.
 - Détruit la racine Tkinter et quitte la fonction.
4. Si un fichier est sélectionné :
 - Instancie `TestAnalyzer` avec le chemin du fichier.
 - Lance l'analyse complète avec `analyzer.analyze()`.
5. Détruit la fenêtre Tkinter à la fin du processus, qu'un fichier ait été sélectionné ou non.

```

def select_and_analyze():
    root = tk.Tk()
    root.withdraw()

    test_file = filedialog.askopenfilename(
        title="S lectionner le fichier de test unitaire",
        filetypes=[("Fichiers Python", "*.py"), ("Tous les fichiers",
            " *.*")]
    )

    if not test_file:

```

```
        messagebox.showwarning("Aucun fichier sélectionné", "Vous  
        devez sélectionner un fichier de test pour continuer.")  
    root.destroy()  
    return  
  
    analyzer = TestAnalyzer(test_file)  
    analyzer.analyze()  
  
    root.destroy()
```

Listing 1.29 – Méthode de sélection des fichiers de tests avec Python

1.4 Conclusion

L'agent présenté dans ce chapitre offre une solution efficace pour évaluer la qualité d'un projet Python sans intervention manuelle. En combinant rigueur syntaxique et analyse contextuelle en utilisant un LLM, il permet d'obtenir une vision claire de l'état de la documentation, de la couverture des tests et de la conformité des commentaires.

Chapitre 2

Évaluation de l'agent

2.1 Introduction

Ce chapitre expose le test de notre agent CodeAnalysisAgent. Cette étape est très intéressante pour plusieurs raisons, par exemple : validation de la précision, compréhension des limites, amélioration de l'agent, confiance des utilisateurs, etc. En résumé, tester cet agent est crucial pour garantir sa fiabilité et une excellente performance. Cela permet à l'utilisateur de vérifier la qualité du code Python et son bon fonctionnement (notamment si les tests unitaires sont valides) avant de procéder à un achat.

2.2 preparation des testes

On a préparé un dossier nommé Testes, qui contient 100 sous-dossiers (T1, T2, ..., T100), comme illustré à la Figure 2.1

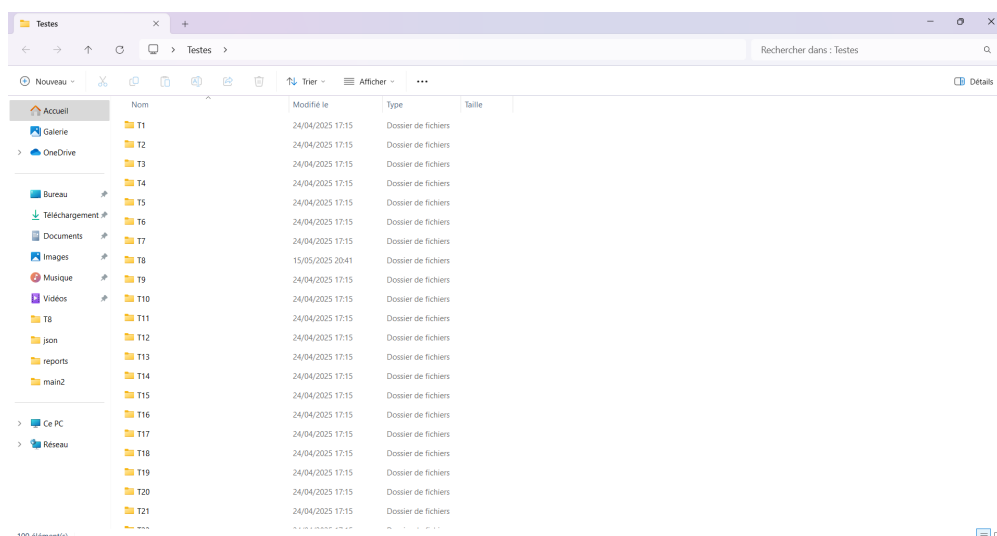


FIGURE 2.1 – Le dossier Testes

Chacun de ces sous-dossiers contient un fichier Python nommé `code.py`, qui contient du code Python (par exemple, 10 fonctions). Certains sous-dossiers contiennent également un autre fichier Python de tests unitaires nommé `test.py`. Ce fichier contient les tests unitaires correspondant au code, comme par exemple le dossier **T1** (voir la figure ci-dessous).

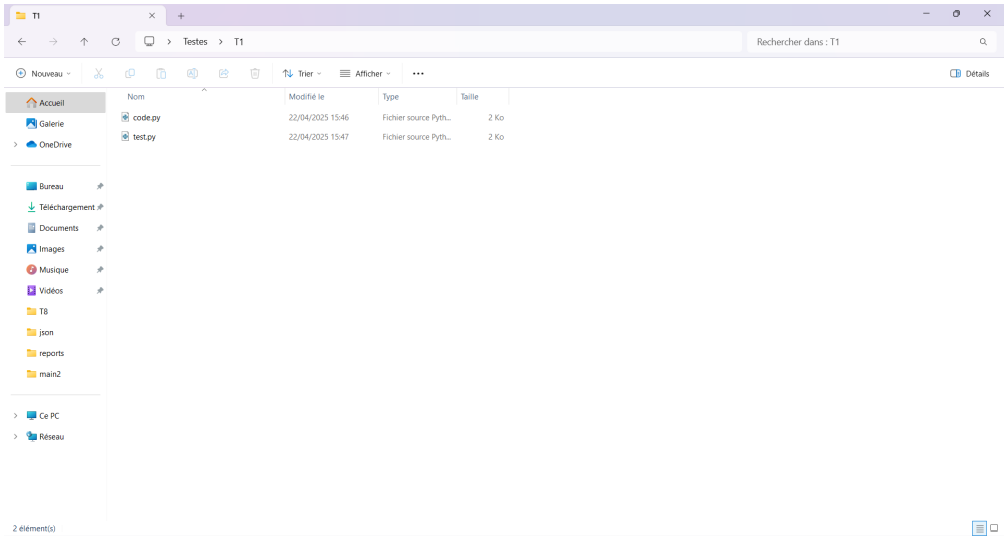


FIGURE 2.2 – Le dossier T1

Nous avons également créé un autre dossier nommé **json**, qui contient 100 fichiers `.json` représentant les caractéristiques de nos 100 dossiers de tests, comme illustré dans la figure ci-dessous

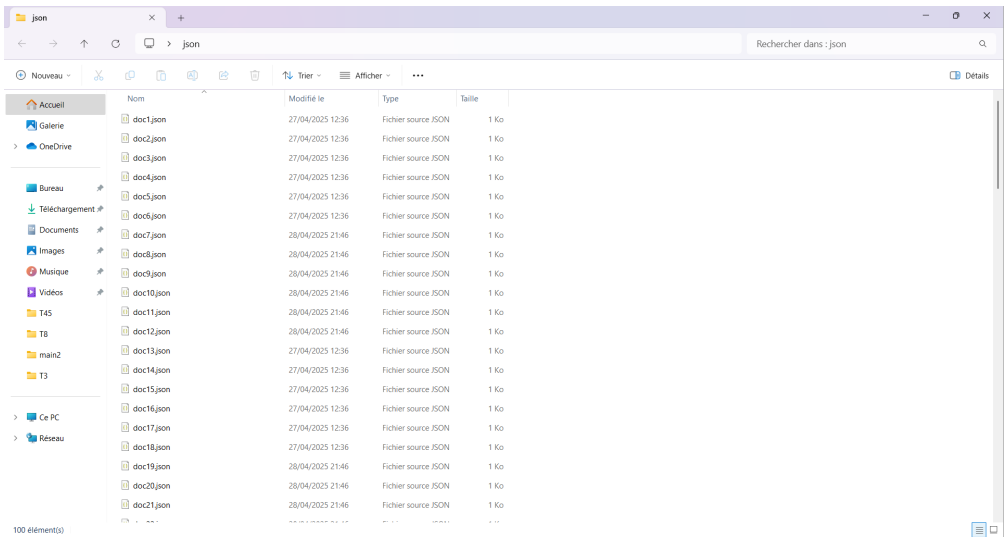


FIGURE 2.3 – Le dossier json

par exemple , `doc1.json` contient les caractéristiques du dossier **T1**, et ainsi de suite.

Nous allons prendre le fichier `doc1.json` comme exemple afin de comprendre et visualiser le contenu d'un fichier `.json` dans le dossier JSON. Les autres fichiers possèdent les mêmes champs et la même structure, seules les valeurs numériques diffèrent. Voir la figure ci-dessous.

```
{
  "all_functions_commented": 0,
  "comments_accurate": 0.1,
  "docstrings_accurate": 0,
  "files_exist": 1,
  "files_valid": 1,
  "fully_documented": 0,
  "has_tests": 1,
  "tests_valid": 0
}
```

FIGURE 2.4 – Le document `doc1.json`

Nous allons expliquer le contenu de notre fichier `.json` :

- `"all_functions_commented"` : Prend la valeur 1 si toutes les fonctions du fichier `code.py` sont commentées (avec des commentaires `#`), sinon 0.
- `"comments_accurate"` : Précision des commentaires (valeur comprise entre 0 et 1 ; plus elle est proche de 1, plus les commentaires sont précis).
- `"docstrings_accurate"` : Précision des docstrings (valeur comprise entre 0 et 1 ; plus elle est proche de 1, plus les docstrings sont précis).
- `"files_exist"` : Tous les fichiers attendus sont présents.
- `"files_valid"` : Tous les fichiers sont valides.
- `"fully_documented"` : Prend la valeur 1 si toutes les fonctions du fichier `code.py` sont documentées (avec des docstrings), sinon 0.
- `"has_tests"` : Prend la valeur 1 si le dossier contient un fichier de test unitaire `test.py` en plus du fichier `code.py`, sinon 0.
- `"tests_valid"` : Prend la valeur 1 si le test unitaire est valide, sinon 0.

Après chaque exécution d'un dossier, notre agent enregistre automatiquement un fichier `.json`, comme celui illustré dans la **Figure 2.4**. Ce fichier contient les caractéristiques du dossier analysé, telles qu'évaluées par l'agent (c'est-à-dire les résultats produits par l'agent).

Tous les fichiers `.json` sont enregistrés dans un dossier nommé `reports`. Nous avons exécuté les 100 dossiers de test ($T_1, T_2, \dots, T_{100}$) un par un. Pour chaque dossier T_i , l'agent a généré un fichier `report_ $T_i$ .json` dans le dossier `reports`, avec $i = 1, \dots, 100$.

voir la figure ci-dessous

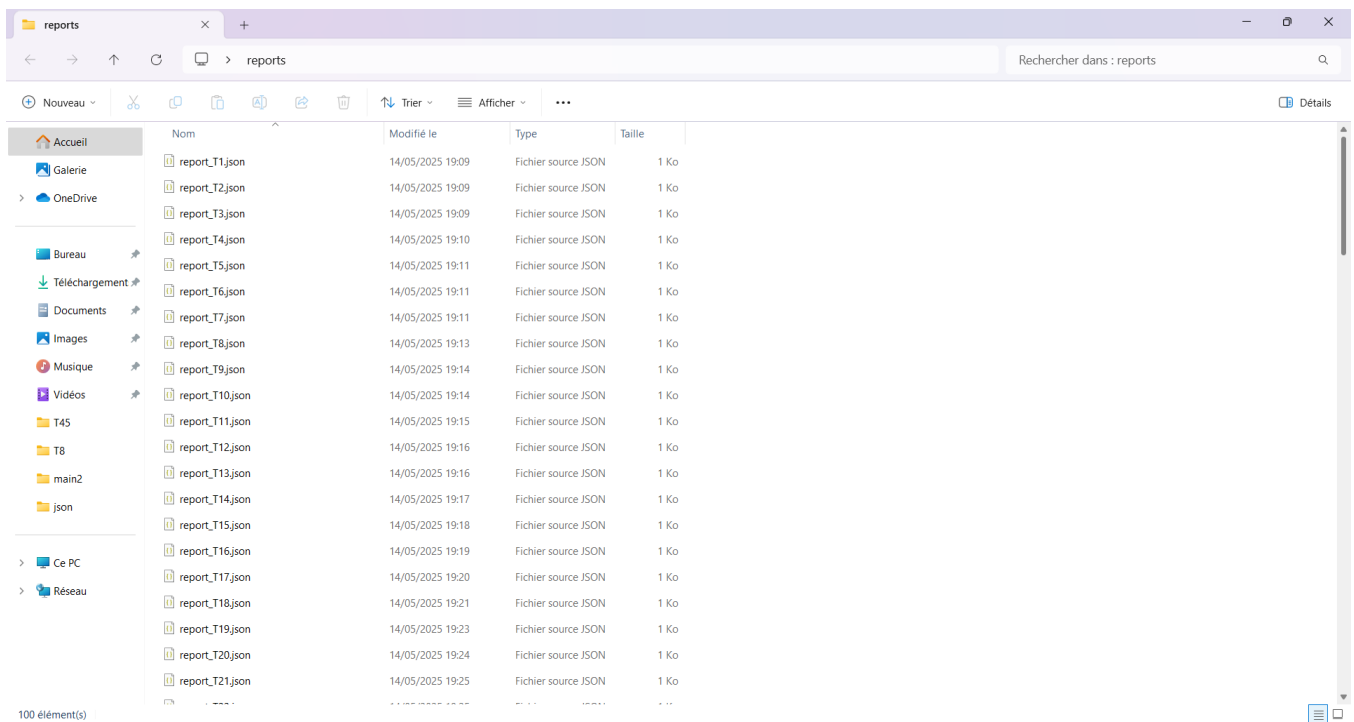


FIGURE 2.5 – Le dossier reports

2.3 Calcul de score

Pour calculer le score, nous avons préparé une fonction appelée `compare_json_files` qui prend en entrée deux dossiers : **json** (contenant 100 fichiers `.json`, chacun représentant les informations réelles d'un dossier T_i) et **reports** (contenant 100 fichiers `.json`, chacun représentant les résultats produits par l'agent pour le dossier T_i).

Cette fonction compare le contenu des deux fichiers `.json` correspondants pour chaque dossier T_i , où $i = 1, \dots, 100$, de manière itérative, puis elle fournit un score global.

```

def compare_json_files(json_folder, reports_folder):
    total_files = 100
    conformes = 0
    for i in range(1, total_files + 1):
        # Ouverture et chargement des deux fichiers JSON
        with open(os.path.join(json_folder, f"doc{i}.json"), 'r') as
            f_json, \
                open(os.path.join(reports_folder, f"report_T{i}.json"),
                    'r') as f_report:
            json_data = json.load(f_json)
            report_data = json.load(f_report)
            all_ok = True
            for key, expected in json_data.items():
                found = report_data.get(key)

                if key in ["comments_accurate", "docstrings_accurate"]:
                    # Pour ces champs numériques avec tolérance d'erreur
                    ( 0.1)
                    if found is None or not (expected - 0.1 <= round(found,
                        3) <= expected + 0.1):
                        all_ok = False
                        break
                else:
                    # Pour les autres champs : galit strict
                    if expected != found:
                        all_ok = False
                        break
            if all_ok:
                conformes += 1
    print(f"score {conformes} = /100")

```

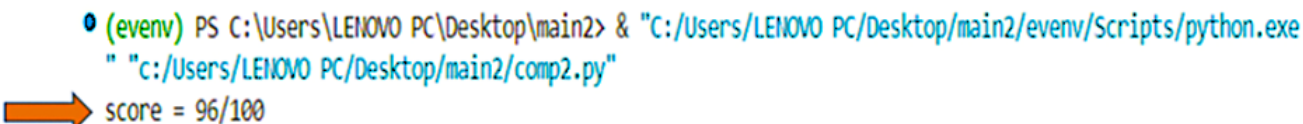
Listing 2.1 – Calcul de score

La fonction `compare_json_files` :

- Prend en entrée deux dossiers :
 - `json_folder` : dossier contenant les fichiers `doc{i}.json`.
 - `reports_folder` : dossier contenant les fichiers `report_T{i}.json`.
- Compare les contenus de 100 paires de fichiers `.json` (`doc1.json` avec `report_T1.json`, ..., `doc100.json` avec `report_T100.json`).
- Vérifie si les paires sont **conformes** selon certaines règles, définies comme suit :

- Pour chaque paire de fichiers `.json` (`doc1.json`, `report_T1.json`), on compare toutes les clés présentes dans le fichier `doc{i}.json`.
- pour la clé est `"comments_accurate"` ou `"docstrings_accurate"` :
 - La valeur dans `report_T{i}.json` peut légèrement différer de celle dans `doc{i}.json` avec Une tolérance d'erreur de ± 0.1
 - `round()` : arrondit à trois décimales la valeur comparée avant la comparaison.
- Pour toutes les autres clés :
 - Une égalité **strictement exacte** est exigée.
- Si toutes les clés respectent ces règles, la paire de fichiers est considérée comme **conforme** et on ajoute 1 la variable **conforme** , sinon 0
- On répète ce principe 100 fois (boucle) pour tester nos cas T1 à T100.
- compte le nombre de fichiers conformes.
- Affiche le score de notre agent , `score=conforme/100`

On fournit les chemins des dossiers en argument de la fonction, on l'exécute, puis le programme affiche le score.



```
(evenv) PS C:\Users\LENOVO PC\Desktop\main2> & "C:/Users/LENOVO PC/Desktop/main2/venv/Scripts/python.exe" "C:/Users/LENOVO PC/Desktop/main2/comp2.py"
score = 96/100
```

FIGURE 2.6 – Le score

Le score obtenu est de 96/100, ce qui signifie que, sur un total de 100 dossiers de test, l'agent a fourni une réponse correcte dans 96 cas. Ce résultat témoigne d'une excellente performance.

2.4 Conclusion

Ce chapitre met en évidence l'importance de tester l'agent, la méthodologie utilisée pour effectuer les tests, ainsi que le processus de calcul du score de l'agent, qui nous permet d'évaluer ses performances.